Posisi Software Testing dan Level Pengujian

Mata Kuliah:
Pengujian Perangkat Lunak (CIF62128)
Disusun oleh:
Software Engineering Lab.

Version: 1.0
Date: 25; Month: February; Year: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Agenda

- Posisi Testing dalam SDLC
- Keterkaitan V-Model
- Level Pengujian
- Artefak Pengujian

Capaian Pembelajaran Pertemuan Ini

1. Mahasiswa mampu menjelaskan posisi pengujian pada setiap fase SDLC secara tepat
2. Mahasiswa mampu memetakan setiap fase pengembangan dengan level pengujian yang sesuai
3. Mahasiswa mampu membedakan tujuan, ruang lingkup, dan peran pada setiap level pengujian
4. Mahasiswa mampu mengidentifikasi deliverables yang dihasilkan pada setiap level pengujian

© Fakultas Ilmu Komputer, Universitas Brawijaya

Software Development Life Cycle (SDLC)

- SDLC adalah kerangka kerja terstruktur yang mendefinisikan **proses**, **aktivitas**, dan **deliverables** dalam **membangun software** berkualitas tinggi. -*Pressman & Maxim (2020, p. 33)*

Fase Umum SDLC:

1. Requirements Analysis
2. System Design
3. Implementation (Coding)
4. **Testing & Verification**
5. Deployment
6. Maintenance

Tujuan SDLC:

- ✔ Memberikan struktur pengembangan
- ✔ Mengontrol kualitas di setiap fase
- ✔ Meminimalkan risiko dan biaya
- ✔ Memastikan produk memenuhi kebutuhan
- ✔ Mempercepat time-to-market

© Fakultas Ilmu Komputer, Universitas Brawijaya

Software Development Life Cycle (SDLC)

Waterfall

Proses sekuensial linear. Setiap fase harus selesai sebelum fase berikutnya dimulai. Cocok untuk proyek dengan requirements stabil (Sommerville, 2016, p. 48).

- ✔ Dokumentasi lengkap, mudah dikelola
- ✘ Tidak fleksibel terhadap perubahan
- ✘ **Testing terlambat** (***late*** *defect detection*)

V-Model

Ekstensi Waterfall yang menekankan verifikasi dan validasi pada setiap fase pengembangan. Setiap fase dev memiliki fase testing yang bersesuaian (Spillner et al., 2014, p. 12).

- ✔ **Deteksi defect lebih awal**
- ✘ Kaku, biaya tinggi jika ada perubahan

Agile

Pengembangan iteratif dan inkremental. Testing terintegrasi dalam setiap sprint/iterasi. Merespons perubahan requirements secara cepat (Beck et al., 2001, Agile Manifesto).

- ✔ Adaptif, feedback cepat
- ✔ **Testing di setiap iterasi**
- ✔ **Automation testing** sangat dianjurkan
- ✔ Tester **kolaborasi** erat dengan Developer
- ✔ **Shift-left**: testing dilakukan dini
- ✘ Dokumentasi kurang formal

© Fakultas Ilmu Komputer, Universitas Brawijaya

Shift Left Testing in SDLC

Shift Left Testing

alphabin

Software Testing Life Cycle (STLC)

Menurut ISTQB Foundation Level Syllabus (2023, p. 17), STLC adalah "*a **sequence** of specific activities conducted during the **testing process** to **ensure software quality goals** are met.*" STLC merupakan **subset dari SDLC** yang berfokus khusus pada aktivitas pengujian.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Software Testing Life Cycle (STLC)

1 Requirement Analysis → 2 Test Planning → 3 Test Case Design → 4 Test Environment Setup → 5 Test Execution → 6 Test Cycle Closure

Fase STLC	Entry Criteria	Aktivitas Utama	Exit Criteria
Requirement Analysis	Requirements doc	Analisis kebutuhan pengujian, identifikasi testable requirements	RTM awal, Automation feasibility
Test Planning	RTM, Requirements	Menyusun Test Plan: scope, strategy, resources, schedule	Test Plan document
Test Case Design	Test Plan, RTM	Membuat Test Cases, Test Scripts, Test Data	Test Cases reviewed & baselined
Test Env. Setup	Test Plan, System Design	Menyiapkan lingkungan uji (hardware, software, network)	Environment ready, Smoke Test pass
Test Execution	Test Cases, Test Env.	Menjalankan test cases, mencatat defect, regression testing	Test Report, Defect Report
Test Cycle Closure	Test Report, Defect data	Evaluasi test coverage, lessons learned, metrics analysis	Test Closure Report

© Fakultas Ilmu Komputer, Universitas Brawijaya

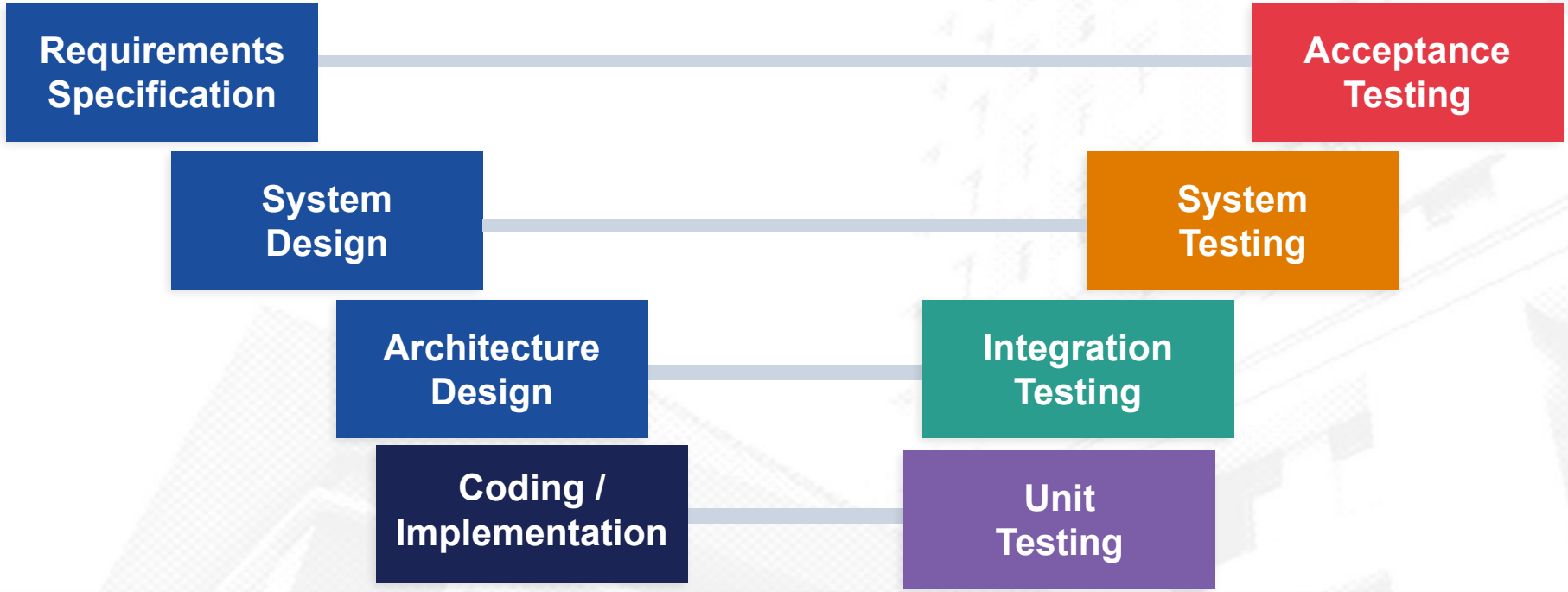
SDLC vs STLC

Aspek	SDLC	STLC
Tujuan	Menghasilkan perangkat lunak berkualitas	Memastikan perangkat lunak bebas defect
Cakupan	Seluruh siklus pengembangan (plan → deploy)	Khusus aktivitas pengujian (subset SDLC)
Pelaku Utama	Project Manager, Developer, Business Analyst	Test Manager, QA Engineer, Tester
Produk	Software + dokumentasi teknis	Test Plan, Test Cases, Test Report, Defect Log
Dimulai Saat	Proyek dimulai (feasibility study)	Requirements tersedia (bisa sejajar SDLC)
Fase (Contoh)	Planning → Analysis → Design → Coding → Testing → Deployment	Req. Analysis → Test Plan → Test Design → Execution → Closure

📌 *Catatan **Penting**:*
*"**Testing** is **not a phase that begins after coding is complete**, but an activity that **should be integrated throughout the development lifecycle**" (Pressman & Maxim, 2020, p. 531).*

© Fakultas Ilmu Komputer, Universitas Brawijaya

V-Model:
Mapping Fase Pengembangan ↔ Level Pengujian



Keterkaitan Fase–Level (Spillner et al., 2014, p. 12):

- Requirements ↔ Acceptance Testing
- System Design ↔ System Testing
- Architecture Design ↔ Integration Testing
- Coding ↔ Unit Testing

Konsep Kunci: Setiap fase pengembangan pada sisi kiri V-Model memiliki fase verifikasi/validasi yang bersesuaian pada sisi kanan. "The V-model makes explicit the relationship between each development phase and its associated testing phase" (Spillner et al., 2014, p. 14). **Defect yang terdeteksi lebih awal memiliki biaya perbaikan yang jauh lebih rendah** (Myers et al., 2011, p. 6).

© Fakultas Ilmu Komputer, Universitas Brawijaya

V-Model: Mapping Detail

Requirement Analysis → Acceptance Testing
- Test berdasarkan kebutuhan bisnis user
- Validasi: "Apakah sistem sesuai kebutuhan?“
- Dilakukan oleh: User / Stakeholder

System Design → System Testing
- Test integrasi keseluruhan system
- Validasi fungsi, performa, keamanan
- Dilakukan oleh: Independent QA Team

Architecture Design → Integration Testing
- Test interaksi antar komponen/modul
- Memastikan interface antar modul benar
- Dilakukan oleh: Developer/QA

Coding / Implementasi → Unit Testing
- Test unit terkecil (fungsi/method)
- Memastikan logika kode benar
- Dilakukan oleh: Develop

© Fakultas Ilmu Komputer, Universitas Brawijaya

Requirement Traceability Matrix (RTM)

- dokumen yang **memetakan** dan **menelusuri kebutuhan pengguna** dengan ***test case*** yang dirancang untuk memverifikasi kebutuhan tersebut.
- IEEE Std 829-2008 menyatakan bahwa *traceability* memastikan **setiap kebutuhan tercakup oleh** minimal **satu *test case***, sehingga tidak ada celah dalam pengujian.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Contoh RTM – Sistem E-Commerce

Req. ID	Deskripsi Requirement	Test Case ID	Deskripsi Test Case	Level Pengujian	Status
REQ-001	Customer dapat melakukan login dengan username & password	TC-001	Verifikasi login berhasil dengan kredensial valid	System Testing	**Pass ✔**
REQ-002	Customer dapat menambah barang ke keranjang belanja	TC-002	Unit test fungsi addToCart()	Unit Testing	**Pass ✔**
REQ-003	Customer dapat melakukan pembayaran yang terintegrasi dengan payment gateway	TC-003	Integrasi modul pembayaran–order processing	Integration Testing	**Fail ✘**
REQ-004	Customer dapat melakukan checkout dan konfirmasi pesanan	TC-004	UAT end-to-end proses pembelian oleh user nyata	Acceptance Testing	**Pending**

© Fakultas Ilmu Komputer, Universitas Brawijaya

RTM: Forward & Backward Traceability

Forward Traceability

- Dari **Requirement → Test Case**
- Memastikan setiap requirement memiliki test case
- Mencegah **Test Gap** (requirement tidak diuji)

Backward Traceability

- Dari **Test Case → Requirement**
- Memastikan setiap test case merujuk ke requirement
- Mencegah **Gold Plating** (test tanpa dasar requirement)

Isi Dokumen RTM:

- ID Requirement
- Deskripsi Requirement
- ID Test Case
- Tipe/Level Pengujian
- Status Eksekusi
- Defect Reference (jika ada)

Tool yang Umum Digunakan:

- Microsoft Excel / Google Sheets
- Jira + Xray
- TestRail, Zephyr

© Fakultas Ilmu Komputer, Universitas Brawijaya

Level 1: Unit Testing

Definisi & Tujuan

Definisi: Unit testing adalah pengujian terhadap komponen perangkat lunak terkecil yang dapat diuji secara independen (unit). Myers et al. (2011, p. 52) mendefinisikan unit sebagai "the smallest testable piece of an application (**method/function/class**)."

Tujuan Utama:
- Memverifikasi logika internal setiap modul
- Mendeteksi defect sedini mungkin dalam siklus pengembangan
- Memudahkan *refactoring* dengan jaminan tidak ada regresi
- Mendokumentasikan perilaku unit secara eksekutabel

Aktor: Developer (bukan tester independen)
Scope: Single method/function atau single class
Framework Populer: JUnit (Java), pytest (Python), NUnit (.NET), Jest (JavaScript)

Contoh Praktis (JUnit – Java)

```
// Kelas yang diuji
public class Calculator {
  public int
add(int a, int b) {
    return a + b;
  }
}

// Unit Test (JUnit 5)
@Test void testAdd() {
  Calculator c = new Calculator();
  assertEquals(5, c.add(2,3));
}
```

Karakteristik Test Case Unit Testing:
- Fast: eksekusi dalam milidetik
- Isolated: tidak bergantung database/network
- Repeatable: hasil konsisten setiap dijalankan
- Self-validating: pass/fail otomatis tanpa inspeksi manual
- Timely: ditulis sebelum/bersamaan kode (TDD)

© Fakultas Ilmu Komputer, Universitas Brawijaya

Unit Testing: Stub, Driver, Mock

Unit Testing sering memerlukan **komponen pengganti** (*test doubles*)

Stub (Penyedia Data Tetap) :

- Menggantikan komponen dependen
- Mengembalikan nilai hardcoded
- Digunakan saat unit bergantung pada modul lain

Driver (Penggerak Tes):

- Menggantikan modul pemanggil (caller)
- Mensimulasikan pemanggilan dari atas
- Digunakan saat modul atas belum tersedia

Mock:

- Versi lebih canggih dari Stub
- Dapat diverifikasi: apakah method dipanggil?
- Digunakan untuk memverifikasi perilaku
- **Contoh:** Mock database, Mock API

Contoh Mockito:

```
when(repo.findById(1)).thenReturn(user);
```

```
verify(repo, times(1)).findById(1);
```


© Fakultas Ilmu Komputer, Universitas Brawijaya

Level 2: Integration Testing

Memverifikasi **interaksi antar unit/modul** yang telah melewati unit testing. Myers et al. (2011, p. 55) menyatakan bahwa integration testing berfokus pada "*finding defects in the interfaces and interactions between integrated components or systems*."

Aktor: Developer + QA Engineer
Tool Java: Spring Test, REST Assured, Testcontainers

© Fakultas Ilmu Komputer, Universitas Brawijaya

Level 2: Integration Testing

Big-Bang Integration

Semua modul diintegrasikan sekaligus lalu diuji. Mudah dilakukan namun sangat sulit mengisolasi defect. Tidak direkomendasikan untuk sistem besar.

✔ Cepat setup
✘ Isolasi defect sangat sulit

Top-Down Integration

Dimulai dari **modul level tertinggi ke bawah**. Modul bawah yang belum siap digantikan stub. Defect di alur utama terdeteksi lebih awal (Pressman & Maxim, 2020, p. 536).

- Mulai dari modul utama (atas)
- Modul bawah diganti Stub

✔ Deteksi masalah arsitektur dini
✘ Butuh banyak stub

Bottom-Up Integration

Dimulai dari **modul level terendah ke atas**. Modul atas yang belum siap digantikan driver. Cocok ketika komponen low-level kritis (Sommerville, 2016, p. 229).

- Mulai dari modul utilitas (bawah)
- Modul atas diganti Driver

✔ Modul kritis diuji awal
✘ UI/Main logic diuji belakangan

Sandwich: Kombinasi top-down & bottom-up

Contoh Praktis: Pada sistem e-commerce, Integration Testing memverifikasi bahwa modul Cart berhasil berkomunikasi dengan modul Payment Gateway dan modul Inventory. Defect yang umum ditemukan: data mapping error, protocol mismatch, timing issues pada API calls.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Level 3: System Testing

Definisi: menguji sistem secara keseluruhan terhadap functional dan non-functional requirements. ISTQB (2023, p. 21) mendefinisikan sebagai "testing of a complete system or product." Dilakukan oleh tim QA independen, bukan developer, dalam lingkungan yang menyerupai produksi.

Functional Testing

Verifikasi bahwa setiap fitur berperilaku sesuai spesifikasi. Berbasis business requirements dan use cases.

Performance Testing

Menguji responsivitas, stabilitas, dan skalabilitas sistem di bawah beban kerja. Mencakup load, stress, dan spike testing.

Security Testing

Mengidentifikasi kerentanan keamanan (OWASP Top 10). Mencakup penetration testing, authentication, authorization.

Usability Testing

Mengevaluasi kemudahan penggunaan antarmuka. Melibatkan pengguna nyata dengan skenario tugas terstruktur.

Compatibility Testing

Memverifikasi sistem berjalan benar di berbagai browser, OS, perangkat, dan resolusi layar.

Regression Testing

Memastikan perubahan/perbaikan tidak merusak fungsionalitas yang sebelumnya bekerja (Pressman & Maxim, 2020, p. 540).

Level 4: Acceptance Testing

Definisi: Acceptance Testing adalah level pengujian terakhir yang dilakukan untuk menentukan apakah sistem memenuhi business requirements dan layak diterima oleh pengguna. Myers et al. (2011, p. 160) menyatakan bahwa acceptance testing adalah "formal testing with respect to user needs, requirements, and business processes."

User Acceptance Testing (UAT)

Pelaku: End User / Client

Tujuan: Memvalidasi bahwa sistem memenuhi kebutuhan bisnis aktual pengguna. Dilakukan berdasarkan real business scenarios, bukan test cases teknis.

Contoh: Tim purchasing perusahaan menguji seluruh alur pembelian: dari PO creation, approval workflow, hingga receipt konfirmasi.

Karakteristik: *Formal, berdasarkan business acceptance criteria yang disepakati di awal proyek*

Alpha Testing

Pelaku: Internal (QA + representative user) di developer site

Tujuan: Pengujian pra-rilis oleh tim internal dalam lingkungan yang dikontrol. Menemukan defect sebelum produk dirilis ke pengguna luar.

Contoh: Tim QA Google menguji versi beta Chrome di kantor Google sebelum dirilis ke kelompok pengguna eksternal terpilih.

Karakteristik: *Lingkungan terkontrol, pelaku internal/terpilih, berlangsung sebelum Beta Testing*

Beta Testing

Pelaku: External users (pengguna nyata di lingkungan mereka)

Tujuan: Pengujian pra-rilis di lingkungan nyata oleh pengguna sesungguhnya. Mendapatkan feedback real-world yang tidak dapat disimulasikan secara internal (Sommerville, 2016, p. 232).

Contoh: Google merilis Chrome Beta kepada publik untuk mendeteksi masalah kompatibilitas di berbagai konfigurasi perangkat nyata.

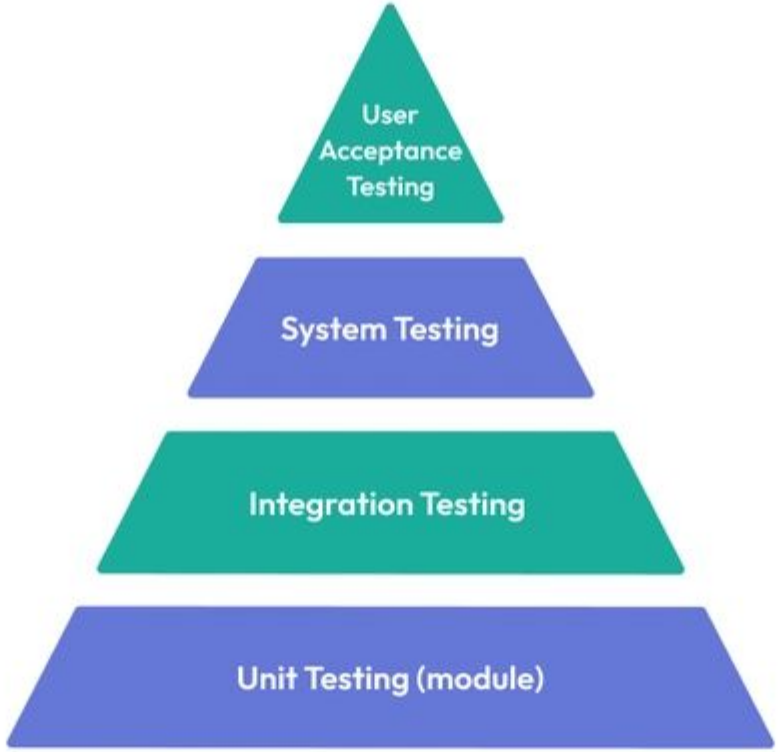
Karakteristik: *Lingkungan tidak terkontrol, feedback pengguna nyata, berlangsung sebelum general release*

Ringkasan Level Pengujian: Tujuan & Peran

Level Pengujian	Objek Uji	Tujuan Utama	Peran Utama	Basis Pengujian
Unit Testing	Modul/fungsi tunggal	Verifikasi logika internal; defect early detection	Developer	Detail design, source code
Integration Testing	Antarmuka antar modul	Verifikasi interaksi dan interface antar komponen	Developer / QA Engineer	Architecture design, API spec
System Testing	Sistem terintegrasi penuh	Verifikasi fungsional & non-fungsional sesuai spec	QA Engineer (independen)	System requirements, use cases
Acceptance Testing	Sistem + business processes	Validasi kesesuaian dengan kebutuhan bisnis pengguna	End User / Client	Business requirements, contractual obligations

Biaya Relatif Perbaikan Defect Berdasarkan Fase Deteksi (Myers et al., 2011, p. 6)

Piramida Pengujian



Prinsip Piramida:
Basis: Unit tests terbanyak (~70%)
Tengah: Integration tests (~20%)
Puncak: E2E/Acceptance tests (~10%)

Anti-pattern: Ice Cream Cone
Terlalu banyak manual/UI test
Sedikit unit test
Lambat, mahal, rapuh
Defect ditemukan terlambat

Peran dalam Setiap Level Pengujian

Developer

- Bertanggung jawab: **Unit Testing**
- Berpartisipasi: Integration Testing
- Menulis test doubles (mock, stub)
- Review test cases QA

QA Engineer / Tester

- Bertanggung jawab: **Integration, System Testing**
- Membuat & mengeksekusi test cases
- Logging & tracking defects
- Laporan hasil pengujian

User / Stakeholder

- Bertanggung jawab: **Acceptance Testing (UAT)**
- Memberikan sign-off untuk release
- Menentukan pass/fail berdasarkan business need

Test Lead / QA Manager

- Membuat Test Plan & strategi
- Koordinasi semua level testing
- Memantau metrics & progress
- Keputusan go/no-go release

© Fakultas Ilmu Komputer, Universitas Brawijaya

Artefak Pengujian: Test Plan & Test Case

📋 Test Plan (IEEE Std 829-2008)

Definisi: Dokumen formal yang mendeskripsikan scope, approach, resources, dan schedule aktivitas pengujian (IEEE Std 829-2008, §5.3.1).

Komponen Wajib:

1. Test Plan Identifier – ID unik dokumen
2. Introduction – latar belakang & tujuan
3. Test Items – sistem/fitur yang diuji
4. Features to be Tested – scope inklusif
5. Features NOT to be Tested – exclusion list
6. Approach – metode & teknik pengujian
7. Item Pass/Fail Criteria – definisi done
8. Suspension/Resumption Criteria
9. Test Deliverables – artefak yang akan dihasilkan
10. Testing Tasks & Schedule
11. Environmental Needs
12. Staffing & Training
13. Risks & Contingencies

📝 Test Case – Contoh Praktis

Field	Isi
TC ID	TC-LOGIN-001
Judul	Login berhasil dengan kredensial valid
Requirement	REQ-001: User Authentication
Precondition	User terdaftar; sistem login aktif
Test Data	user: admin@test.com / pw: Test@123
Steps	1. Buka /login 2. Isi email & password 3. Klik 'Login'
Expected Result	Redirect ke dashboard; session terbentuk
Actual Result	[Diisi saat eksekusi]
Status	Pass / Fail / Blocked
Priority	High
Test Level	System Testing

Artefak Pengujian:
Test Script · Test Report · Defect Report

⚙️ Test Script

Definisi:
Implementasi eksekutabel dari test case, umumnya ditulis dalam bahasa pemrograman atau framework automation (ISTQB, 2023, p. 25).

```
def test_login_success(driver):
  driver.get('/login')
  driver.find_element(
    By.ID,'email'
  ).send_keys(
    'admin@test.com')
  driver.find_element(
    By.ID,'password'
  ).send_keys('Test@123')
  driver.find_element(
    By.ID,'btn-login').click()
  assert '/dashboard'
    in driver.current_url
```

Framework populer:
Selenium, Cypress, Playwright, Appium, Robot Framework, JMeter (performance)

📊 Test Report

Komponen	Deskripsi
Summary	Total TC: Pass/Fail/Skip/Blocked
Test Coverage	% requirements tercakup test
Defect Summary	Jumlah defect per severity
Test Execution Log	Detail hasil per test case
Risks & Issues	Hambatan selama testing
Rekomendasi	Go/No-Go deployment decision

Contoh Metrik (TC: 100 total):

78%	12%	6%	4%
Pass	Fail	Blocked	Skip

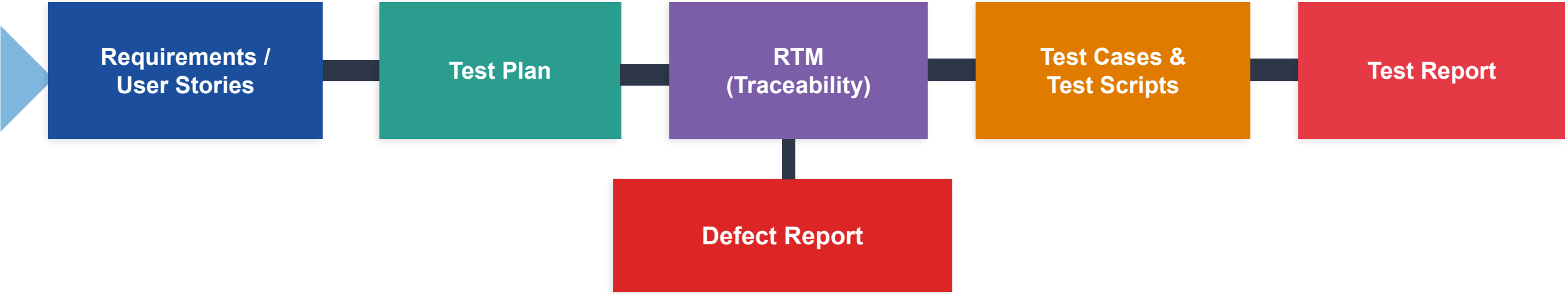
Disampaikan kepada: Test Manager, Project Manager, Client.
Berbasis IEEE Std 829-2008, 5.3.6.

🐛 Defect Report

Field	Isi
Defect ID	BUG-2025-0042
Judul	Login gagal dengan email valid
Severity	Critical
Priority	High
Ditemukan Saat	System Testing – Sprint 5
Steps to Reproduce	1.Buka/login 2.Masukkan admin@test.com 3.Klik Login
Expected	Redirect ke /dashboard
Actual	Error 500: Internal Server Error
Assigned To	Dev Team – Backend
Status	Open → In Progress

Severity Levels:
Critical → High → Medium → Low
(Kaner et al., 1999, p. 105)

Hubungan Antar-Artefak dalam Siklus Pengujian



📋 Test Plan

Fase: Test Planning | Test Manager

Mendefinisikan strategi pengujian, scope, sumber daya, jadwal, dan kriteria exit. Menjadi fondasi seluruh aktivitas pengujian. Dibuat berdasarkan SRS dan Project Plan.

IEEE Std 829-2008, 5.3.1

📝 Test Case

Fase: Test Case Design | QA Engineer

Mendokumentasikan kondisi pengujian spesifik: input, langkah-langkah, expected result, dan kriteria pass/fail. Setiap test case memetakan minimal satu requirement dalam RTM.

Myers et al. (2011, p. 66)

⚙️ Test Script

Fase: Test Design (jika automation) | Automation Engineer

Kode program yang mengotomatiskan eksekusi test case. Memungkinkan regression testing efisien setiap ada perubahan kode. Bersifat reusable dan maintainable.

ISTQB (2023, p. 25)

📊 Test Report

Fase: Test Execution & Closure | Test Manager

Merangkum hasil eksekusi: coverage, pass/fail rate, defect density, dan rekomendasi Go/No-Go. Menjadi dasar keputusan release oleh Project Manager dan stakeholder.

IEEE Std 829-2008, 5.3.6

Q&A

Referensi

- Ammann, P., & Offutt, J. (2016). Introduction to Software Testing (2nd ed.). Cambridge University Press.
- Aniche, M. (2022). Effective Software Testing: A Developer's Guide. Manning Publications.
- Graham, D., van Veenendaal, E., Evans, I., & Black, R. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning.
- Perry, W. E. (2006). Effective Methods for Software Testing (3rd ed.). Wiley Publishing, Inc.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

© Fakultas Ilmu Komputer, Universitas Brawijaya